

FRED TALKS

1st October 2026

CONTENTS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 2024 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player [Brainstorm](#) app.

4. I have long believed that Claude “feels better” to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askill’s work on its “soul”). My guess is that if you tried to make a “less human” version of Claude, it would become rapidly less capable.

↩

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though [not all](#)) think that human identities are narratively constructed.

↩

3. I wrote about this last year in [Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality](#). A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

↩

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. [Lace](#) got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's [Lethal Trifecta](#) - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally

- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses [superpowers-chrome](#) to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called [Slackline](#) - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · [SOURCE](#)

Get started with Claude Managed Agents by following our [docs](#).

A running topic on the Engineering Blog is how to [build effective agents](#) and [design harnesses](#) for [long-running work](#). A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can [go stale](#) as models improve.

As just one example, in prior work [we found](#) that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called “context anxiety.” We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for “[programs as yet unthought of](#).” Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.

Process tool results in code with programmatic tool calling

[Programmatic tool calling](#) processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly [37%](#) on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

[Skills and MCP are complementary](#). MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

[Plugins](#) for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

Elicitation lets your server pause mid-tool call to ask the user for input. Form mode sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. URL mode hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest MCP spec supports CIMD (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. Vaults in Claude Managed Agents covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

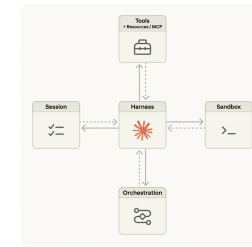
Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the “brain” (Claude and its harness) from both the “hands” (sandboxes and tools that perform actions) and the “session” (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died,

the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (methods)	Satisfiability
Session	<code>getSession(session_id)</code> → {events, timeLeft} <code>getSession(session_id)</code> → {sessionId, nextEvent} <code>emitEvent(id, event)</code>	Any agent only logging that can be accessed at any time, any event logs and events stream support— Pipelines, IDs, in-memory errors, etc.
Orchestration	<code>wake(session_id)</code> → void	Any scheduler that can call a function with any ID and can follow a cron job or a periodic schedule, or other long jobs.
Harness	<code>createEffectivity()</code> → Effectiveness	Any logic that can effect and depends on agents in the harness.
Sandbox	<code>provision({resources})</code> <code>executeCommand, input()</code> → {output}	Any system that can be configured and executed with any command and a set of input/output, or other container, etc.
Resource	<code>getSession_id, nextEvent()</code>	Any resource that can be accessed and fetched from by reference— Pipelines, IDs, log streams, etc.
Tools	<code>Tools, nextEvent()</code> <code>Input, output()</code>	Any capability that can be accessed and used to input/output— MCP servers, custom tools, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in [prior work](#) on context engineering. For example, compaction lets Claude save a

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

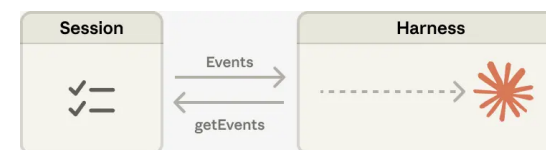
Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work [has explored](#) ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

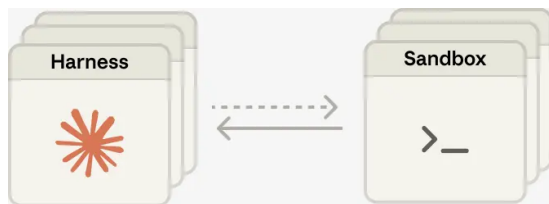
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call (`execute(name, input) → string`) only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work—a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.